10 Python Programming Problem Statements

with File Handling

Concepts Covered:

Variables · Datatypes · Literals · Base Conversions · Type Conversions · Arithmetic · Bitwise · Logical · Comparison
if / if-else / if-elif-else · match-case · for · while · break · continue · pass
Lists · Tuples · Sets · Dictionaries · String Methods · List Comprehension
*Functions: Positional-only · Keyword-only · Keyword Args · Positional Args · Mixed · *args*
Exception Handling (try-except) · File Handling: r/w/a/rb/wb/r+ · Context Manager · seek/tell · CSV · JSON

Table of Contents

01	Personal Diary & Journal System
02	Student Marks Database with CSV Storage
03	Inventory Tracker with JSON Persistence
04	Cipher Toolkit with File I/O
05	Library Management System with File Persistence
06	Word Frequency Analyser & Report Generator
07	Scientific Calculator with History & Config Files
08	Multi-File Log Analyser & Anomaly Detector
09	Contact & Address Book with vCard Export
10	Population Census Simulator — Capstone

Concept Coverage Matrix

Concept	Covered In
variables & datatypes	**All 10**
literals (int, float, bool, hex 0x, bin 0b)	**All 10**
base conversions (bin, oct, hex, int(,base))	#4, #7, #10
type conversions	**All 10**
arithmetic operators	**All 10**
bitwise operators (&, |, ^, ~, <<, >>)	#3, #4, #7, #8, #10
logical operators (and, or, not)	**All 10**
comparison operators	**All 10**
if condition	**All 10**
if-else condition	**All 10**
if-elif-else condition	**All 10**
match-case	**All 10**
lists & list methods	**All 10**
list comprehension	**All 10**
tuples	**All 10**
sets & set methods	**All 10**
dictionaries & dict methods	**All 10**
for loop	**All 10**
while loop	**All 10**
break	**All 10**
continue	**All 10**
pass	**All 10**
string methods (ALL major methods)	**All 10**
functions — positional-only args (/)	**All 10**
functions — keyword-only args (*)	**All 10**
functions — keyword arguments	**All 10**
functions — positional arguments	**All 10**
functions — mixed (/ and *)	**All 10**
functions — *args (var-positional)	#1, #2, #6, #7, #8, #10
exception handling — try-except	**All 10**
file handling — read ('r')	**All 10**

file handling — write ('w')	**All 10**
file handling — append ('a')	**All 10**
file handling — binary ('rb'/'wb')	#1, #3, #4, #8, #10
file handling — read-write ('r+')	#4, #7, #10
file handling — context manager (with)	**All 10**
file handling — seek() & tell()	#4, #6, #7, #8, #10
file handling — CSV (manual)	#2, #5, #9, #10
file handling — JSON/structured text	#3, #10
file handling — os.path / file ops	#1, #5, #6, #8, #9

Problem
#01 Personal Diary & Journal System

Concepts: variables | datatypes | literals | type conversion | arithmetic | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only keyword args | exception handling | try-except | file handling | file read/write | context manager

Build a personal diary application that creates, reads, updates, and searches journal entries stored in plain text files — one file per month. Every concept from function arguments to file seek is exercised naturally.

File Handling Focus: file handling · file read/write · context manager

Key Tasks to Implement:

1	create_entry(date, title, body, /, *, mood='neutral', tags=None) — positional-only date+title+body; keyword-only metadata. Serialize as a structured text block and append to YYYY-MM.txt using 'a' mode inside a with statement.
2	read_entries(filepath, /) — positional-only; open file in 'r' mode with try-except FileNotFoundError; parse each entry block into a dict; return list of dicts.
3	Use string methods: strip(), split('\n'), split('---'), startswith(), find(), replace(), lower() to parse raw text into structured entry dicts.
4	search_entries(*keywords, month=None, mood=None, case_sensitive=False) — var-positional keywords + keyword-only filters; use list comprehension with logical operators (and, or, in) to match entries.
5	Store all unique tags across entries in a set; use set union (|) across monthly files to build master tag list.
6	Entry statistics dict: {mood: count}. Use arithmetic to compute percentages; if-elif-else to classify overall wellbeing (positive/neutral/negative).
7	Use match-case on mood string: 'happy','sad','anxious','excited','neutral' → assign emoji label and color code (as a string).
8	while True menu loop: break on 'quit', continue on invalid option, pass as stub for 'export to PDF' feature.
9	backup_diary(src_dir, dest_dir, /, *, compress=False, overwrite=False) — mixed; use os.path and open() in binary 'rb'/'wb' mode to copy files.
10	List comprehension: [e['title'] for e in entries if mood in e['mood'] and any(k in e['body'].lower() for k in keywords)].
11	Type conversion: int() for year/month from filename, str() for date parts, bool() for flags, len() for word count (arithmetic).
12	Wrap all file operations in try-except (FileNotFoundError, PermissionError, IOError) with descriptive messages using f-strings and string methods.

Problem
#02 Student Marks Database with CSV Storage

Concepts: variables | datatypes | literals | arithmetic | type conversion | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | csv files | context manager

Manage student academic records stored as CSV files. Support bulk import, grade computation, rank generation, and subject-wise analytics — all persisted to and read from disk with full exception safety.

File Handling Focus: file handling · csv files · context manager

Key Tasks to Implement:

1	load_csv(filepath, /) — positional-only; open in 'r' mode with newline=''; use manual split(',') with strip() to parse; return list of dicts. Raise ValueError if header row is missing; catch with try-except.
2	save_csv(records, filepath, /, *, append=False, include_header=True) — mixed; open in 'a' or 'w' mode; use join(',') + '\n' to write each row; wrap in with statement.
3	add_student(student_id, name, scores_dict, /) — positional-only; validate with if-else and logical operators; compute GPA using arithmetic (sum / count); append to CSV.
4	compute_grade(score, /) — positional-only; if-elif-else with comparison operators: >=90→A+, >=80→A, >=70→B, >=60→C, >=40→D, else→F.
5	Subject analytics: build dict {subject: [scores]}; for each subject compute min, max, average using arithmetic and for loop.
6	Use sets to detect duplicate student IDs when loading multiple CSV files: seen_ids set with add() and 'in' check.
7	generate_report(*student_ids, format='detailed', include_rank=True) — var-positional + keyword-only; write formatted report to a .txt file in 'w' mode.
8	String methods: zfill(6) for ID padding, title() for names, ljust()/rjust() for table alignment, center() for headers, format() for GPA with 2 decimal places.
9	match-case on format arg: 'detailed','summary','csv','json' — writes different output layouts to different file extensions.
10	List comprehension: [(s['name'], compute_grade(avg)) for s in students if float(s['gpa']) >= 7.0] — top performer extraction.
11	while True interactive menu; break on '0'; continue on malformed CSV row; pass as placeholder for email-report feature.
12	Type conversion throughout: int() for IDs, float() for scores/GPA, str() for formatted output; all wrapped in try-except ValueError.

Problem
#03 Inventory Tracker with JSON Persistence

Concepts: variables | datatypes | literals | bitwise | arithmetic | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | mixed arguments | exception handling | try-except | file handling | json files | context manager | file modes

Build a warehouse inventory system that persists all data as a JSON file. Bitwise flags track product status, sets handle tag management, and all CRUD operations are file-backed with full exception handling.

File Handling Focus: file handling · json files · context manager · file modes

Key Tasks to Implement:

1	load_inventory(filepath, /) — positional-only; open 'r' with try-except FileNotFoundError; parse JSON manually using split and strip (no json module) OR read raw text and eval safely; return dict.
2	save_inventory(inventory, filepath, /, *, indent=2, backup=True) — mixed; if backup=True copy existing file to .bak first using 'rb'/'wb' binary modes; write formatted JSON string using string methods (join, format, replace).
3	add_product(sku, name, price, /, qty=0, *, category='general', flags=0b0001, reorder=10) — positional-only identity, keyword-only policy; validate types with isinstance() and logical operators; raise TypeError in try-except.
4	Bitwise flags: bit0=active, bit1=perishable, bit2=imported, bit3=fragile. Use & to check, | to set, ^ to toggle, ~ to clear.
5	stock_update(sku, delta, /, *, reason='manual', log_file='stock_log.txt') — mixed; append tuple (sku, delta, reason, timestamp_str) to log file in 'a' mode; update qty with arithmetic; if-elif-else for reorder alert.
6	Use sets for category management: all_categories = {p['category'] for p in inventory.values()}; set operations for filtering.
7	search_products(*keywords, category=None, in_stock_only=True, min_price=0.0) — var-positional + keyword-only; list comprehension with logical operators.
8	String methods: upper() for SKU normalisation, strip() on all inputs, ljust()/rjust() for report columns, zfill(8) for numeric IDs, f-strings for JSON-like output.
9	match-case on stock_update reason: 'sale','return','damage','restock','adjustment' — update separate counters dict and write summary to stats.txt.
10	Base conversion: write product flags in binary (bin()), octal (oct()), hex (hex()) to a debug log file for audit trail.
11	List comprehension: [{'sku':k,'name':v['name'],'qty':v['qty']} for k,v in inventory.items() if v['qty'] <= v['reorder'] and (v['flags'] & 1)].
12	Wrap entire save/load cycle in try-except (FileNotFoundError, ValueError, KeyError, PermissionError); use finally to ensure file handle closure even without 'with'.

Problem
#04 Cipher Toolkit with File I/O

Concepts: variables | datatypes | literals | base conversion | bitwise | type conversion | arithmetic | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | binary files | file seek/tell | context manager

Implement a file-encryption toolkit supporting Caesar, XOR, and Vigenère ciphers. Read plaintext from files, encrypt/decrypt, and write output back — with binary mode for XOR and seek/tell for partial encryption.

File Handling Focus: file handling · file read/write · binary files · file seek/tell · context manager

Key Tasks to Implement:

1	encrypt_file(src_path, dest_path, cipher, key, /) — positional-only; open src in 'r', dest in 'w' (text ciphers) or 'rb'/'wb' (XOR); wrap both in with; raise FileNotFoundError caught by caller.
2	caesar_cipher(text, key, /, direction='encrypt') — positional-only text+key; shift with arithmetic (ord(c)-65+key)%26; use isalpha(), isupper(), islower() string methods.
3	xor_encrypt(data_bytes, key_int, /) — positional-only; apply ^ bitwise XOR on each byte (int); use bytes() and list comprehension; write result to file in 'wb' mode.
4	vigenere(text, keyword, *, repeat=True, strict=False) — keyword-only options; cycle keyword with index%len(keyword) arithmetic; use for loop and string methods.
5	Use file seek() and tell() to encrypt only a specific byte range: open in 'r+b', seek to offset, read chunk, XOR-encrypt, seek back, write. Wrap in try-except.
6	Store cipher history as list of tuples (filename, cipher_name, key_hint, timestamp); persist to history.txt in 'a' mode with join() serialisation.
7	Dictionary of cipher stats: {cipher_name: {'files_processed': int, 'bytes_total': int}}; save/load from stats.json as formatted text.
8	String methods: translate(), maketrans(), encode('utf-8'), decode(), upper(), lower(), replace() for pre-processing plaintext.
9	match-case on cipher name string: 'caesar','xor','vigenere','rot13','base64' — dispatch to respective function; default raises NotImplementedError in try-except.
10	Base conversion: display each character's ordinal as decimal, binary (bin()), and hex (hex()) in a debug output file.
11	List comprehension: [chr(b ^ key_int) for b in bytearray(plaintext.encode())] — XOR all chars in one expression.
12	while REPL loop; break on 'exit'; continue on bad file path; pass as stub for RSA encryption feature.

Problem
#05

Library Management System with File Persistence

Concepts: variables | datatypes | literals | arithmetic | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | csv files | context manager | os module

A full library management system where books, members, and lending records are stored in separate text/CSV files. Every operation reads from and writes to disk, with complete exception handling and file safety.

File Handling Focus: file handling · file read/write · csv files · context manager · os module

Key Tasks to Implement:

1	load_books(filepath, /) — positional-only; open 'r'; parse CSV manually with split(','), strip(); return dict {isbn: book_dict}; try-except FileNotFoundError returns empty dict.
2	save_books(books_dict, filepath, /) — positional-only; open 'w' with context manager; write header line then each book using join(',') and string methods; try-except PermissionError.
3	add_book(isbn, title, author, /, copies=1, *, genre='General', tags=None) — positional-only identity, keyword-only policy; validate isbn with isdigit() and len(); append to books.csv.
4	issue_book(member_id, isbn, /, *, due_days=14) — mixed; read both members.csv and books.csv; validate with if-else; append lending record tuple to lending.txt in 'a' mode.
5	compute_fine(member_id, isbn, return_date, /, *, rate=2.0) — mixed; read lending.txt to find issue_date; arithmetic for days overdue * rate; if-else for grace period; log fine to fines.txt.
6	Use sets: available_isbns = all_isbns - set(issued_isbns); set operations across files to find multi-borrowed books.
7	search_books(*, title=None, author=None, genre=None, available_only=False) — all keyword-only; load books.csv; list comprehension with logical operators for multi-field match.
8	String methods: lower() for case-insensitive search, strip() on every parsed field, title() for display, ljust()/center() for table formatting, f-strings for file output.
9	match-case on menu command: 'add','issue','return','search','report','fine','exit' — routes to functions; all wrapped in outer try-except.
10	generate_report(*, report_type='overdue', output_file='report.txt') — all keyword-only; open output_file in 'w'; write multi-section formatted report using string methods.
11	List comprehension: [(m_id, isbn, days) for (m_id, isbn, issued, due) in records if (today - due_date(due)) > 0] — overdue computation.
12	while True main loop; break on 'exit'; continue on invalid ISBN; pass as stub for QR-scan integration; all I/O wrapped in try-except.

Problem
#06

Word Frequency Analyser & Report Generator

Concepts: variables | datatypes | literals | arithmetic | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | context manager | os module

Read one or multiple text files, perform deep linguistic analysis (word frequency, sentence stats, readability, n-grams), and write detailed reports back to disk — all string methods and file modes exercised.

File Handling Focus: file handling · file read/write · context manager · os module

Key Tasks to Implement:

1	load_text(filepath, /, *, encoding='utf-8', lowercase=True) — mixed; open in 'r' with specified encoding in try-except (UnicodeDecodeError, FileNotFoundError); apply lower() if flag set.
2	Apply ALL string methods during parsing: strip(), split(), splitlines(), replace(), translate(), maketrans(), find(), rfind(), count(), startswith(), endswith(), isalpha(), isdigit(), isspace(), upper(), lower(), title(), center(), ljust(), rjust(), zfill(), join(), encode(), expandtabs().
3	build_frequency(words, /, *, exclude_stopwords=True, min_length=2) — mixed; build dict {word: count} using for loop and dict.get(); filter with list comprehension and logical operators.
4	compute_ngrams(words, n, /) — positional-only; build list of tuples using list comprehension: [tuple(words[i:i+n]) for i in range(len(words)-n+1)].
5	Use sets: unique_words = set(all_words); stopwords_set for O(1) lookup; hapax_legomena = {w for w,c in freq.items() if c == 1}.
6	Statistics dict: {word_count, sentence_count, avg_word_len, avg_sentence_len, lexical_density}; all computed with arithmetic operators.
7	analyse_files(*filepaths, merge=False, report_dir='.') — var-positional + keyword-only; process each file; if merge=True combine all word lists; write per-file and combined report.
8	write_report(stats, filepath, /, *, format='text', top_n=20) — mixed; open 'w'; write formatted sections using string methods; match-case on format: 'text','csv','markdown'.
9	if-elif-else for readability classification: avg_sentence_len < 10 → 'Simple', < 20 → 'Moderate', < 30 → 'Complex', else → 'Very Complex'.
10	Use file seek(0) and tell() to get file size before reading; display as formatted string with arithmetic (size // 1024 for KB).
11	List comprehension: [(word, count, round(count/total*100, 2)) for word, count in sorted(freq.items(), key=lambda x:-x[1])[:top_n]].
12	while loop for batch processing; break on empty input; continue on non-.txt file; pass as placeholder for NLP integration; all wrapped in try-except.

Problem
#07

Scientific Calculator with History & Config Files

Concepts: variables | datatypes | literals | base conversion | type conversion | arithmetic | bitwise | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | context manager | file seek/tell

A feature-rich calculator REPL that saves calculation history to a log file, persists user preferences to a config file, and supports base-conversion and bitwise operations — all file-backed and exception-safe.

File Handling Focus: file handling · file read/write · context manager · file seek/tell

Key Tasks to Implement:

1	load_config(filepath='calc.cfg', /) — positional-only with default; open 'r' with try-except; parse KEY=VALUE lines using split('=', 1) and strip(); return dict with typed values via type conversion.
2	save_config(config_dict, filepath, /) — positional-only; open 'w'; write each key-value pair using f-strings and join(); wrap in context manager.
3	evaluate(expression_str, /, *, precision=6, degree_mode=False) — mixed; parse operators from string using split() and strip(); compute with arithmetic; raise ValueError for invalid expression; catch ZeroDivisionError.
4	log_calculation(expression, result, /, *, log_file='history.log', timestamp=True) — mixed; append formatted line to log in 'a' mode; use string methods: ljust(), format(), center().
5	read_history(log_file, /, *, last_n=10, filter_keyword=None) — mixed; open 'r'; read all lines; use list slicing [-last_n:] and list comprehension with string 'in' operator to filter.
6	base_convert(value, from_base, to_base, /) — positional-only; use int(str, base) with try-except ValueError; convert to target with bin(), oct(), hex(); log result to file.
7	bitwise_ops(a, b, /, op='and') — positional-only operands, keyword op; match-case: 'and'→&, 'or'→|, 'xor'→^, 'lshift'→<<, 'rshift'→>>; log with type conversion (str(result)).
8	statistics(*values, ddof=0, log_results=True) — var-positional + keyword; compute mean, std dev, median without imports using arithmetic; optionally write to stats.txt.
9	Use seek(0) and read() to load entire history file at once; use tell() before each write to record byte position for indexed lookup.
10	match-case on REPL command: 'calc','convert','bitwise','stats','history','config','clear','exit' — dispatches to functions.
11	Set of used operations tracked per session: session_ops.add(op_name); display at exit; persist to session_summary.txt in 'w' mode.
12	List comprehension: [line.strip() for line in history_lines if filter_keyword in line.lower()] — filtered history display.

Problem
#08 Multi-File Log Analyser & Anomaly Detector

Concepts: variables | datatypes | literals | arithmetic | bitwise | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | binary files | context manager | os module | file seek/tell

Ingest, parse, and analyse multiple log files. Detect anomalies using statistical thresholds, generate alert reports, and produce daily summaries — combining every file mode, all string methods, and full exception safety.

File Handling Focus: file handling · file read/write · binary files · context manager · os module · file seek/tell

Key Tasks to Implement:

1	parse_log_line(line, /, *, strict=False) — mixed; use split(), strip(), find(), startswith() to extract timestamp, level, message; return tuple (ts_str, level, msg) or raise ValueError in try-except.
2	load_logs(*filepaths, encoding='utf-8', skip_errors=True) — var-positional + keyword; open each 'r'; for lines use continue on parse error if skip_errors, else re-raise; return flat list of tuples.
3	Use sets: error_messages = {msg for ts,lvl,msg in entries if lvl=='ERROR'}; unique_sources, unique_days — set operations across multiple log files.
4	Frequency dict {level: count}: build with for loop; use arithmetic for percentages; if-elif-else to classify log health: <5% error → 'Healthy', <15% → 'Warning', else → 'Critical'.
5	detect_anomalies(entries, /, *, threshold=2.0, window_minutes=5) — mixed; compute mean and std dev of event counts per window using arithmetic; flag windows > mean + threshold*std_dev.
6	write_alert_report(anomalies, output_path, /, *, append=False) — mixed; open in 'a' or 'w' mode; format each anomaly tuple as a structured line using string methods: center(), ljust(), rjust(), f-strings.
7	Use bitwise on log-level integer codes: DEBUG=0b0001, INFO=0b0010, WARN=0b0100, ERROR=0b1000; filter using & operator: entries with level & 0b1100 to get WARN+ERROR.
8	match-case on log level string: 'DEBUG','INFO','WARNING','ERROR','CRITICAL' — assign numeric priority and write to priority-sorted output file.
9	Seek operations: open large log in 'rb' binary mode; use seek(0,2) for file end, tell() for file size; seek to middle for binary search of timestamp.
10	generate_daily_summary(*log_dirs, output_dir='.', date_filter=None) — var-positional + keyword; scan all .log files in dirs; aggregate stats; write one summary file per day.
11	List comprehension: [(ts, msg) for ts,lvl,msg in entries if lvl in {'ERROR','CRITICAL'} and any(kw in msg.lower() for kw in alert_keywords)].
12	Wrap all file operations in try-except (FileNotFoundError, PermissionError, UnicodeDecodeError, OSError); use finally for cleanup; pass as stub for email-alert integration.

Problem
#09 Contact & Address Book with vCard Export

Concepts: variables | datatypes | literals | arithmetic | type conversion | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | context manager | csv files | binary files

A complete contact management system that stores contacts in CSV, exports to vCard format (.vcf), imports from plain text, and generates formatted printable directories — all file-backed with complete argument style coverage.

File Handling Focus: file handling · file read/write · context manager · csv files · binary files

Key Tasks to Implement:

1	add_contact(name, phone, /, email='', *, group='General', birthday=None, notes='') — positional-only name+phone; keyword-only metadata; validate phone with isdigit() and len(); append to contacts.csv in 'a' mode.
2	load_contacts(filepath, /) — positional-only; open 'r'; parse CSV manually with split(',', maxsplit=5) and strip(); return list of dicts; try-except FileNotFoundError returns [].
3	export_vcf(contacts, output_path, /) — positional-only; open 'w'; write vCard 3.0 format using string methods (upper(), replace(), join()); wrap each contact block in try-except.
4	import_from_txt(filepath, /, *, delimiter=':', encoding='utf-8') — mixed; open 'r'; parse line-by-line using split(delimiter, 1) and strip(); build dict per contact block.
5	search_contacts(*keywords, group=None, has_email=False, has_birthday=False) — var-positional + keyword-only; list comprehension with logical operators and string methods: lower(), in operator.
6	Use sets: all_groups = {c['group'] for c in contacts}; set operations to merge contacts from multiple CSV files without duplicates (use phone as unique key in a set).
7	Statistics: build dict {group: count}; use arithmetic for percentages; if-elif-else to flag groups with < 3 contacts as 'sparse'.
8	String methods: strip(), split(), title() for name normalisation, zfill(10) for phone padding, ljust()/rjust() for directory table, center() for section headers, f-strings for vCard lines.
9	generate_directory(contacts, output_file, /, *, sort_by='name', group_filter=None, columns=2) — mixed; open 'w'; write formatted printable directory using string methods; use match-case on sort_by.
10	match-case on export format: 'vcf','csv','txt','json_like' — write different file formats; default raises ValueError in try-except.
11	backup_contacts(src, dest_dir, /) — positional-only; open src in 'rb', write to dest_dir/contacts_YYYYMMDD.bak in 'wb'; use arithmetic for date string construction; try-except PermissionError.
12	List comprehension: [f"{c['name']},{c['phone']},{c['email']}" for c in contacts if c['group']==group_filter] — filtered CSV row generator.

Problem
#10 Population Census Simulator — Capstone

Concepts: variables | datatypes | literals | base conversion | arithmetic | bitwise | type conversion | comparison | logical | if-elif-else | match-case | for | while | break | continue | pass | lists | tuples | sets | dictionaries | string methods | list comprehension | functions | positional arguments | keyword arguments | only positional args | only keyword args | mixed arguments | exception handling | try-except | file handling | file read/write | binary files | csv files | json files | context manager | os module | file seek/tell

The capstone problem — a full population data platform covering every concept on the syllabus. Ingest census CSV files, simulate growth, detect trends, and produce multi-format reports — all file-backed, fully exception-safe, and using every function argument style.

File Handling Focus: file handling · file read/write · binary files · csv files · json files · context manager · os module · file seek/tell

Key Tasks to Implement:

1	load_census(filepath, /) — positional-only; open 'r'; parse CSV manually; return list of region dicts {region_id(hex), name, population(int), area(float), growth_rate(float), flags(int), categories(set)}. Try-except (FileNotFoundError, ValueError).
2	save_census(regions, filepath, /, *, mode='w', backup=True) — mixed; if backup copy to .bak in 'rb'/'wb' binary mode; write CSV in specified mode; use join() and string methods; context manager.
3	add_region(region_id, name, population, area, /, growth_rate=0.01, *, decade=2020, flags=0b0001) — positional-only core, keyword-only config; use 0x hex literals for IDs; validate with isinstance() and comparison operators.
4	Bitwise flags: bit0=urban, bit1=coastal, bit2=border, bit3=capital. Check with &, set with |, toggle with ^, clear with & ~mask. Write flag audit trail to flags_log.txt in 'a' mode.
5	simulate_growth(region, years, /, *, model='exponential', log_file=None) — mixed; compute pop=P*(1+r)^t with ** arithmetic; if-elif-else for linear/exponential/logistic; if log_file write each year's data to file in 'a' mode.
6	Base conversion: write each region_id in decimal, binary (bin()), octal (oct()), hex (hex()) to a cross-reference file; use type conversion int(hex_str, 16) when reading back.
7	filter_regions(*flag_bits, min_pop=0, max_density=None, categories=None) — var-positional flag bits + keyword-only thresholds; list comprehension with bitwise checks and logical operators.
8	Use sets: urban_set = {r['name'] for r in regions if r['flags'] & 1}; coastal_set similarly; set intersection/difference/union for multi-criteria analysis; persist set to a .txt file with join().
9	String methods for file output: zfill(8) for IDs, center()/ljust() for table columns, format(pop, ',') for comma-separated numbers, f-strings for JSON-like output, upper() for region names, encode() for binary write.
10	Use seek(0) to rewind after initial read; tell() to record byte offsets of each region's record for random-access lookup in large census files. Open in 'r+' mode for in-place updates.
11	generate_report(regions, /, *, sections=('summary','growth','density'), output_format='text', output_dir='.') — mixed; match-case on output_format: 'text','csv','markdown' — write to .txt/.csv/.md respectively using appropriate string methods.
12	compare_regions(region_a_id, region_b_id, /, *, metrics=('population','density','growth'), report_file=None) — mixed; load from CSV; compute differences with arithmetic; write comparison table to report_file in 'w' mode if provided.
13	List comprehension: [(r['name'], r['population'], r['population']//r['area']) for r in regions if r['flags'] & 1

	and r['population'] > 1_000_000] — mega-city density extraction.
14	while simulation_running: break on convergence or user 'stop'; continue to skip inactive regions; pass as stub for GIS map export; all file operations wrapped in try-except (IOError, OSError, ValueError, KeyError).
15	Full modular design: load_census(), save_census(), add_region(), simulate_growth(), filter_regions(), detect_anomalies(), generate_report(), compare_regions(), backup_data() — each function with explicit argument contracts and docstring.